AIST4010 / ESTR4140 · Foundation of Applied Deep Learning

Lecture 10: Optimization and Loss Functions

Instructor: LI Yu Scribe: LIU Xinran 14 October 2026

1 Optimization methods

1.1 Mini batch SGD and gradient accumulation

- **Notation:** w^t denotes the parameters, L the loss, and α the learning rate. Use initial parameters w^1 and update indices $t \geq 2$; running estimates start at index 0.
- **SGD:** use a mini batch gradient as a cheap, noisy estimate of the full gradient. Noise may help leave poor regions, but does not guarantee a global minimum.

$$\partial_{w^{t-1}} L = \frac{\partial L}{\partial w^{t-1}}, \qquad w^t = w^{t-1} - \alpha \frac{\partial L}{\partial w^{t-1}}.$$

- **Gradient accumulation:** hold the weights fixed for K micro batches. For weight $w_{ij}^{(k)}$ in layer k, accumulate the loss derivatives and update once:
- Y_B and d_B are batch predictions and targets. Equal micro batches of size b give effective size Kb; divide the accumulated amount by K to match a mean batch gradient.

$$\Delta w_{ij}^{(k)} = \alpha \sum_{s=1}^{K} \frac{\partial \mathrm{Div}(Y_{B_s}, d_{B_s})}{\partial w_{ij}^{(k)}}, \quad w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - \Delta w_{ij}^{(k)}.$$

1.2 Momentum and Nesterov accelerated gradient

- **Momentum:** remember the previous displacement $\Delta w^{t-1} = w^{t-1} - w^{t-2}$ (initialize $\Delta w^1 = 0$). Consistent signs reinforce motion; alternating signs partly cancel. Typical $\beta = 0.9$.

$$w^t = w^{t-1} - \alpha \frac{\partial L}{\partial w^{t-1}} + \beta \Delta w^{t-1}.$$

- **NAG:** first extend the previous step by $\beta \Delta w^{t-1}$, then evaluate the loss derivative at that lookahead position. Add the lookahead displacement and gradient correction together.

$$w^t = w^{t-1} - \alpha \frac{\partial L}{\partial (w^{t-1} + \beta \Delta w^{t-1})} + \beta \Delta w^{t-1}.$$

1.3 RMSprop adapts each coordinate

- **Intuition:** repeatedly large gradients create a large RMS scale and smaller effective steps. Squaring prevents sign cancellation; it is **not** a second derivative.
- **Running average:** $\mathrm{E}[(\partial_w L)^2]_{t-1}$ estimates recent squared derivatives. Store one statistic per parameter and initialize it at zero; all operations are elementwise.
- **Typical values:** $\alpha = 0.001$, $\gamma = 0.9$, $\varepsilon = 10^{-7}$. The positive ε stabilizes division near zero.

$$\mathrm{E}[(\partial_w L)^2]_{t-1} = \gamma \mathrm{E}[(\partial_w L)^2]_{t-2} + (1-\gamma)(\partial_{w^{t-1}} L)^2.$$

$$w^t = w^{t-1} - \frac{\alpha}{\sqrt{\mathrm{E}[(\partial_w L)^2]_{t-1} + \varepsilon}} \partial_{w^{t-1}} L.$$

1.4 Adam and bias correction

- **Intuition:** combine momentum like smoothing with RMSprop scaling: the signed derivative average determines the direction, while the squared derivative average sets the coordinate scale.
- **Initialization:** both running estimates start at index 0 with value zero. The second estimate is a raw moment; the variance equals $\mathrm{E}[(\partial_w L)^2] - \mathrm{E}[\partial_w L]^2$.
- **Typical values:** $\alpha = 0.001$, $\delta = 0.9$, $\gamma = 0.999$, $\varepsilon = 10^{-7}$. The positive ε is placed inside the square root.

Running estimates

- For constant α and zero initial displacement, momentum expands into an exponentially weighted sum. Adam maintains exponential moving averages of the first and second raw moments:

$$w^t = w^{t-1} - \alpha \sum_{i=1}^{t-1} \beta^{t-1-i} \partial_{w^i} L.$$

$$\mathrm{E}[\partial_w L]_{t-1} = \delta \mathrm{E}[\partial_w L]_{t-2} + (1-\delta)\partial_{w^{t-1}} L.$$

$$\mathrm{E}[(\partial_w L)^2]_{t-1} = \gamma \mathrm{E}[(\partial_w L)^2]_{t-2} + (1-\gamma)(\partial_{w^{t-1}} L)^2.$$

Why the estimates need correction

- **Generic EMA:** let g_t be the t-th observation and m_t its running average, with $m_0 = 0$. For observations with common mean μ, weights summing to $1 - \delta^t$ create an initial bias toward zero.
- **Correction:** divide by the accumulated weight mass for each estimate. This removes initialization bias; it does not remove the lag caused by changing gradient statistics.
- **Largest effect early:** index $t-1$ contains $t-1$ observations. At the first update ($t = 2$), divide the two estimates by $1 - \delta$ and $1 - \gamma$. The correction factors approach 1 as training continues.

$$m_t = \delta m_{t-1} + (1-\delta) g_t = (1-\delta) \sum_{i=1}^{t} \delta^{t-i} g_i.$$

$$\mathrm{E}[m_t] = (1-\delta) \sum_{i=1}^{t} \delta^{t-i} \mu = (1-\delta^t)\mu.$$

$$\hat{\mathrm{E}}[\partial_w L]_{t-1} = \frac{\mathrm{E}[\partial_w L]_{t-1}}{1-\delta^{t-1}}, \quad \hat{\mathrm{E}}[(\partial_w L)^2]_{t-1} = \frac{\mathrm{E}[(\partial_w L)^2]_{t-1}}{1-\gamma^{t-1}}.$$

Final Adam update

- Use the corrected first moment in the numerator and the corrected RMS scale in the denominator. Backpropagation computes the derivatives; Adam determines the parameter update.

ADAM FINAL UPDATE FORMULA

$$w^t = w^{t-1} - \frac{\alpha}{\sqrt{\hat{\mathrm{E}}[(\partial_w L)^2]_{t-1} + \varepsilon}} \hat{\mathrm{E}}[\partial_w L]_{t-1}.$$

2 Learning rate scheduling

- **Purpose:** use larger steps for early progress and smaller late steps for refinement. Adaptive coordinate scaling does not replace a global learning rate schedule.
- **Optimizer choice:** Adagrad, AdaDelta and AdaMax are related alternatives. Carefully tuned SGD can also work well; compare validation performance under a suitable schedule.

2.1 Factor and multi step decay

- **Factor decay:** multiply the rate by $0 < \tau < 1$ at each scheduled event, optionally enforcing a lower bound:

$$\alpha_{t+1} = \max(\alpha_{\min}, \tau\alpha_t).$$

- **Multi step decay:** decrease the rate only at selected milestones. If $k(t)$ milestones have been reached,
- A traditional choice is to divide the rate by 10 every chosen number of epochs. Tune the factor and milestones using validation results.

$$\alpha_t = \alpha_0 \tau^{k(t)}.$$

2.2 Cosine decay and warmup

- **Cosine decay:** smoothly decrease from α_0 to α_T over a training horizon T:

$$\alpha_t = \alpha_T + \frac{\alpha_0 - \alpha_T}{2}\left(1 + \cos\frac{\pi t}{T}\right), \qquad 0 \le t \le T.$$

- **Warmup:** start with a small rate when early updates are unstable, increase it gradually, and then apply decay. Linear warmup over T_w steps is one example:

$$\alpha_t = \alpha_{\text{start}} + (\alpha_{\text{peak}} - \alpha_{\text{start}})\frac{t}{T_w}, \qquad 0 \le t \le T_w.$$

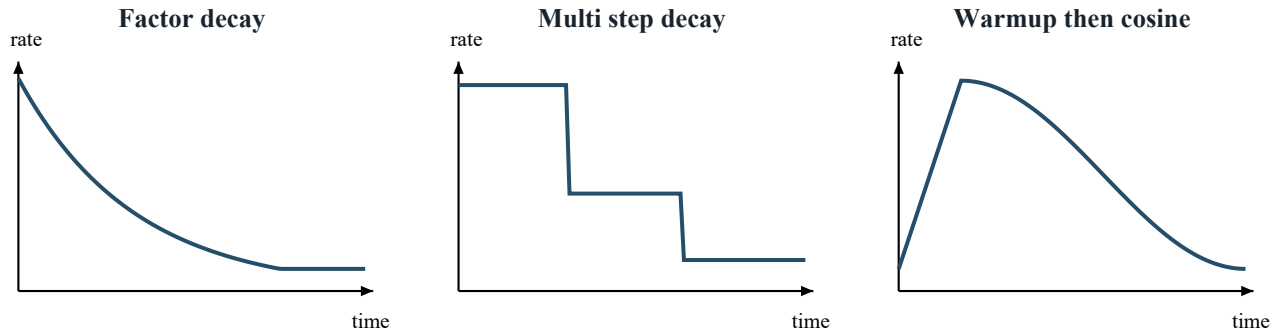


Figure 1. Three schematic schedules: gradual decay, milestone decay, and warmup followed by cosine decay.

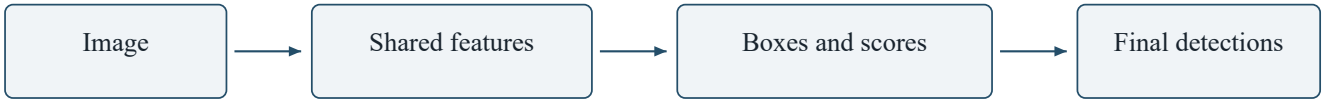
- **Time unit:** specify epochs or optimizer updates. With accumulation, an update based schedule advances once per parameter update, rather than once per micro batch.

3 Designing a deep learning method

- **Check feasibility:** identify the available data, annotations and computing resources.
- **Define the task:** specify the desired output, choose a model, and design a loss aligned with the goal.
- **Verify learning:** first confirm that the model can fit training examples; then improve generalization, evaluate on held out data, and consider deployment.

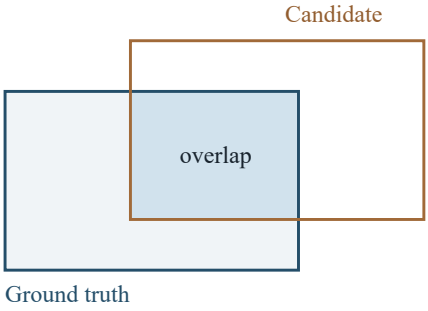
4 Object detection

- **Goal:** identify the objects and their positions. Output multiple boxes, each with a label and confidence score.
- **Inference:** generate candidates → predict classes, scores and offsets → retain confident boxes and suppress duplicates. Keep detections for different objects, rather than only one box per image.



4.1 IoU and confidence

$$\mathrm{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$



IoU = intersection area / union area

0: no overlap 1: exact match

Reference boxes are known during training.

- **IoU:** measures localization quality in [0, 1]. The intersection counts once in the union. At inference the ground truth is unavailable, so the detector predicts a confidence or quality score; IoU is not automatically a class probability.

4.2 Bounding box regression and loss design

- **Offset targets:** for candidate a and matched target b, let x, y be center coordinates and w, h be dimensions. Regress normalized shifts and log scale ratios:

$$t_x = \frac{x_b - x_a}{w_a}, \quad t_y = \frac{y_b - y_a}{h_a}, \quad t_w = \log\frac{w_b}{w_a}, \quad t_h = \log\frac{h_b}{h_a}.$$

- Targets may be standardized as $(t_k - \mu_k)/\sigma_k$. Localization loss applies to matched positive candidates; classification also distinguishes background.

$$L = L_{\mathrm{cls}} + \lambda_{\mathrm{loc}} L_{\mathrm{loc}} + \lambda_{\mathrm{conf}} L_{\mathrm{conf}}.$$

- This is a schematic objective: include a separate confidence term only when the architecture predicts that target. The weights balance the loss components.

4.3 Accuracy and efficiency

- **Share features:** compute a backbone feature map once. Faster R-CNN proposes regions from shared features and then classifies and refines them.
- **Control candidates:** choose scales and aspect ratios, or learn proposals. Class wise non maximum suppression keeps high scores and removes overlapping duplicates.
- **Tradeoffs:** more candidates improve coverage but increase cost; pixel annotations support segmentation. Development spans hand designed detectors, R-CNN variants, YOLO/SSD/RetinaNet, and later CenterNet and DETR.

5 Neural style transfer

• **Goal:** synthesize an image *x* with the structure of content image *c* and the appearance of style image *s*.

5.1 A visual example

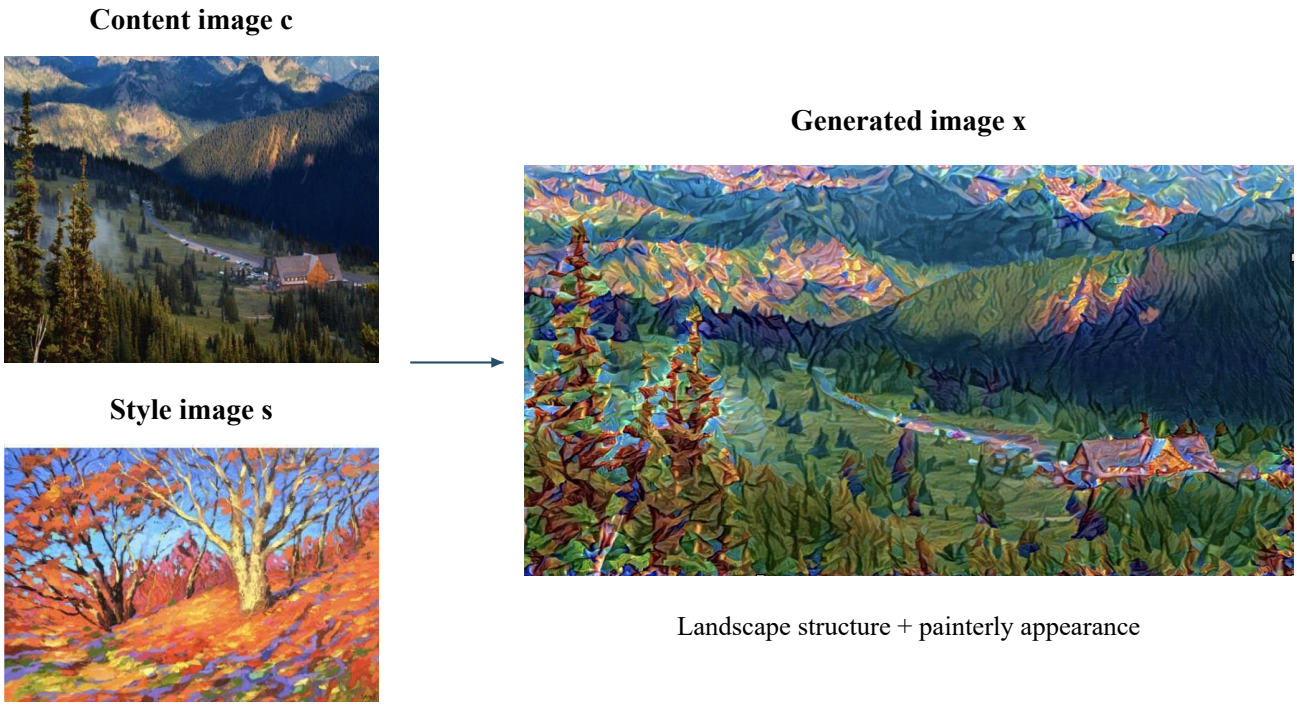


Figure 2. A landscape photograph provides the content, a painting provides the style, and the result combines the landscape with painterly colors and textures.

What each image contributes

• **Content image:** provides the scene structure - the mountains, valley, trees, road and building, together with their spatial arrangement.

• **Style image:** provides visual appearance, including the color palette, brushstroke patterns and texture statistics. Its objects need not align with the content scene.

• **Generated image:** keeps the recognizable landscape while changing its surface appearance. It is synthesized by balancing content and style constraints.

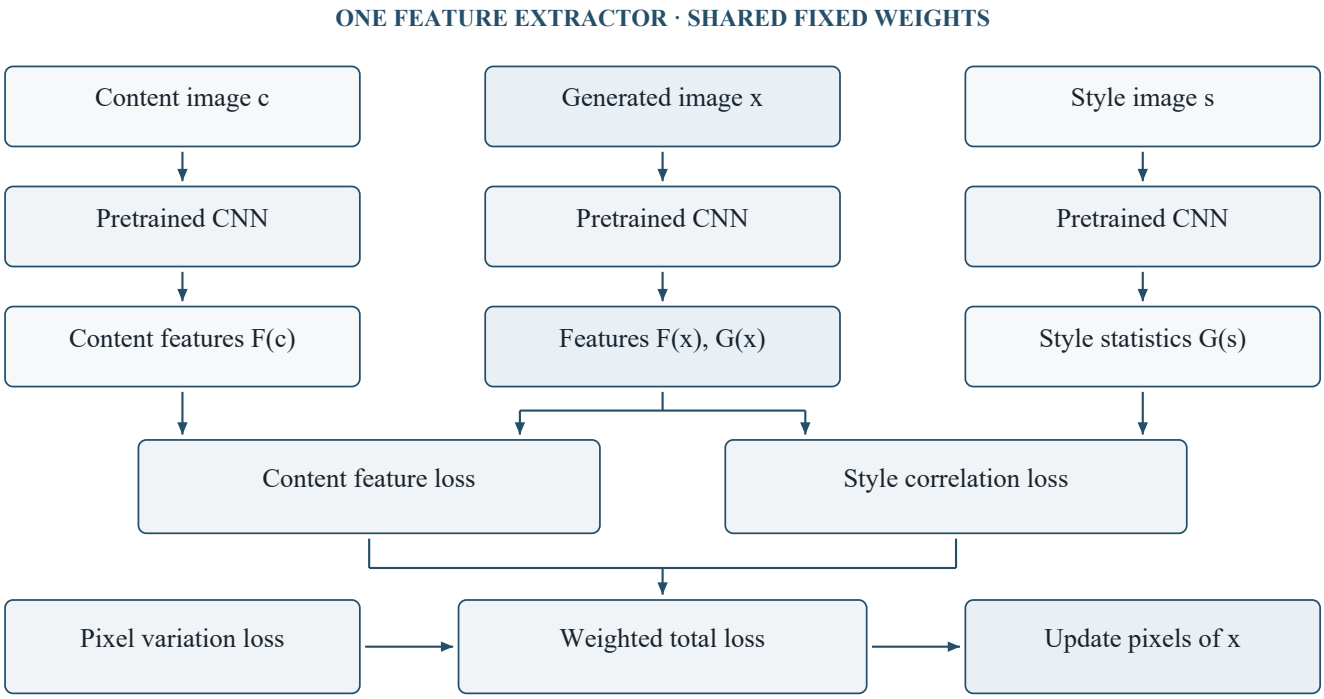
5.2 Optimize the generated image

• **Fixed feature extractor:** use one pretrained CNN to represent all three images; its learned weights remain fixed.

• **Trainable variables:** the pixels of *x*. Initialize them from noise or the content image, compute the losses, and backpropagate through the CNN to update the pixels.

• **Iteration:** repeat feature extraction and pixel updates until the chosen stopping criterion. Loss weights determine the balance between preserving structure and adopting style.

5.3 Feature comparisons and loss functions



Repeat feature extraction and pixel updates; the CNN stays fixed.

Figure 3. Compare content features and style statistics separately, combine the losses, and update the generated image.

Content style and variation losses

- **Content:** compare feature maps at a selected layer ℓ_c to preserve structure. Here $\|\cdot\|_F$ denotes the Frobenius norm.

$$L_{\text{content}} = \| F_{\ell_c}(x) - F_{\ell_c}(c) \|_F^2.$$

- **Style:** reshape features into channels × spatial positions. Gram matrices compare channel correlations, capturing texture without requiring spatial alignment:

$$G_\ell(x) = F_\ell(x) F_\ell(x)^{\mathrm{T}}, \quad L_{\text{style}} = \sum_\ell a_\ell \| G_\ell(x) - G_\ell(s) \|_F^2.$$

- **Variation:** penalize abrupt changes between neighboring pixels. Combine the terms with nonnegative weights:

$$L(x) = \lambda_c L_{\text{content}} + \lambda_s L_{\text{style}} + \lambda_v L_{\text{TV}}.$$

- **Procedure:** initialize x from noise or the content image; extract features, evaluate losses, backpropagate to pixels, and repeat $x \leftarrow x - \eta \nabla_x L(x)$. Content and style may use different layers.
- **Broader picture:** iterative style transfer optimizes each output image; other approaches train a model to produce stylized outputs in a forward pass.